CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
UNIDADE CURRICULAR: QUALIDADE DE SOFTWARE

GUIA DE ATIVIDADE PRÁTICA

Aplicação de Métricas de Qualidade no Projeto Integrador

Pesquisa, diagnóstico técnico e pitch executivo de qualidade | até 10 minutos

1. Contexto e justificativa da atividade

No ecossistema de inovação e startups, uma solução tecnológica sustentável precisa combinar velocidade de entrega, confiabilidade técnica e capacidade de evolução. Apresentar uma proposta de negócio ou um Produto Mínimo Viável (MVP) a investidores, parceiros ou clientes sem evidências sobre a qualidade do software pode aumentar os riscos operacionais, os custos de retrabalho e a insatisfação dos usuários.

Nesta atividade, cada equipe investigará a base de código e o fluxo de trabalho do próprio Projeto Integrador (PI). O objetivo é transformar dados coletados no projeto em indicadores que apoiem a tomada de decisão técnica e gerencial, relacionando qualidade de software, custos e planejamento.

Mindset empreendedor: o objetivo não é punir a existência de defeitos, mas compreender a situação atual do produto, apresentar evidências e propor ações realistas de melhoria.

2. Dinâmica de execução e cronograma de trabalho

A atividade será desenvolvida em três etapas: pesquisa e coleta de dados na aula atual; consolidação dos resultados no intervalo entre as aulas; e apresentação executiva na próxima aula.

Etapa	Atividades previstas	Tempo/prazo
Etapa 1 – Aula atual: pesquisa e extração	• Alinhar a equipe ao repositório e ao quadro de tarefas do PI. • Pesquisar os conceitos de métricas de produto e de processo. • Coletar dados sobre tamanho do código, defeitos, testes e horas/entregas. • Preencher o Quadro de Diagnóstico Técnico.	Durante a aula atual, em sala ou laboratório.
Etapa 2 – Consolidação e preparação do pitch	• Conferir os dados e registrar critérios e limitações da coleta. • Calcular o custo estimado do retrabalho. • Definir ações de melhoria para as próximas sprints. • Elaborar até cinco slides para a apresentação.	Intervalo entre a aula atual e a próxima aula.
Etapa 3 – Apresentação executiva	• Apresentar o diagnóstico de qualidade do PI. • Defender os indicadores e explicar suas limitações. • Justificar as ações propostas e responder à banca.	Até 10 minutos por equipe, mais 3 minutos de arguição.

3. Orientações gerais para coleta dos dados

- Utilizem, preferencialmente, dados reais do repositório, das ferramentas de testes e do quadro de tarefas do PI.
- Registrem a data da coleta, o período analisado, as ferramentas utilizadas e os critérios adotados.
- Não contabilizem como defeitos as funcionalidades ainda não implementadas, sugestões de melhoria ou tarefas pendentes.
- Na contagem de linhas de código (LOC), informem se foram consideradas linhas físicas ou linhas não vazias/de código. Usem o mesmo critério em todo o projeto.
- Excluam, quando possível, bibliotecas de terceiros, dependências, arquivos gerados automaticamente, código minificado e arquivos de configuração que não representem código desenvolvido pela equipe.
- Se não houver dados históricos suficientes, utilizem estimativas justificadas e identifiquem claramente que os valores são estimados, não medidos.
- Não comparem equipes ou projetos diferentes sem considerar escopo, tecnologia, complexidade, período e critérios de medição.

4. Roteiro prático: quatro métricas obrigatórias

Cada equipe deverá pesquisar o conceito, coletar os dados primários e calcular as quatro métricas a seguir. Para cada resultado, expliquem o que foi medido, como foi calculado, o que significa e quais limitações devem ser consideradas.

Métrica 1 – Densidade de defeitos (métrica de produto)

O que pesquisar: o que representa a densidade de defeitos? Por que é importante distinguir defeitos encontrados durante o desenvolvimento, nos testes, na homologação e em produção?

Como aplicar no PI: levantar o total de defeitos efetivamente registrados e o tamanho do código analisado. Um defeito deve ser um problema identificado e registrado, e não uma funcionalidade pendente ou uma melhoria desejada. Sempre que possível, classifiquem os defeitos por severidade e pelo ambiente em que foram encontrados.

KLOC = LOC ÷ 1.000 | Densidade de defeitos = Total de defeitos ÷ KLOC

Exemplo: 4.800 LOC correspondem a 4,8 KLOC. Se houver 24 defeitos registrados: 24 ÷ 4,8 = 5 defeitos/KLOC.

Conexão com o negócio: o indicador ajuda a discutir a estabilidade do MVP e o risco de usuários encontrarem falhas. A densidade, isoladamente, não determina a qualidade: considerem também a severidade dos defeitos, a cobertura do sistema analisado e o período de coleta.

Métrica 2 – Cobertura de testes automatizados (métrica de produto)

O que pesquisar: o que é cobertura de código (Code Coverage)? Como ferramentas como Jest, PyTest e PHPUnit estimam a proporção do código executada durante os testes?

Como aplicar no PI: executar a suíte de testes automatizados disponível no repositório e registrar o relatório da ferramenta. Informar explicitamente qual tipo de cobertura foi utilizado:

- Line Coverage: percentual de linhas executadas pelos testes.
- Branch Coverage: percentual de ramificações/caminhos de decisão executados.
- Function Coverage: percentual de funções executadas.

Cobertura de linhas (%) = (Linhas cobertas ÷ Total de linhas consideradas) × 100

Exemplo: se 2.400 de 4.800 linhas consideradas forem cobertas, a cobertura de linhas será 50%.

Interpretação didática para esta atividade (não é um padrão universal): abaixo de 70% – atenção; de 70% a 80% – faixa intermediária; acima de 80% – boa cobertura. A equipe deve justificar a meta adequada ao risco e ao contexto do projeto.

Atenção: cobertura elevada não garante testes eficazes nem ausência de defeitos. É necessário avaliar também a qualidade dos casos de teste, os cenários verificados e os riscos não cobertos.

Conexão com o negócio: a cobertura pode oferecer evidências sobre quais partes do sistema são exercitadas pelos testes e apoiar decisões de manutenção e evolução. Não deve ser apresentada como garantia de que novas funcionalidades não causarão falhas.

Métrica 3 – Taxa de retrabalho (métrica de processo)

O que pesquisar: o que é retrabalho em processos de desenvolvimento? Como distinguir a correção de problemas de atividades de manutenção evolutiva, melhoria ou implementação de novos requisitos?

Como aplicar no PI: estimar ou extrair do histórico de tarefas o tempo dedicado a refazer atividades ou corrigir problemas decorrentes de trabalho anterior inadequado. Não classifiquem automaticamente toda manutenção ou correção de requisito como retrabalho. Definam e registrem o critério utilizado.

Taxa de retrabalho (%) = (Horas de retrabalho ÷ Total de horas trabalhadas) × 100

Exemplo: 45 horas classificadas como retrabalho em um total de 180 horas trabalhadas resultam em 25% de taxa de retrabalho.

Se as horas forem estimadas, indiquem a fonte da estimativa e reconheçam essa limitação. Conexão com o negócio: o indicador permite discutir quanto do esforço da equipe foi destinado a refazer trabalho e como isso pode afetar prazo, custo e capacidade de entrega.

Métrica 4 – Produtividade da equipe de desenvolvimento (métrica de processo)

O que pesquisar: quais são os cuidados ao medir produtividade em software? Por que contar apenas linhas de código pode levar a conclusões equivocadas?

Como aplicar no PI: escolham uma das abordagens abaixo, de acordo com os dados que a equipe possui. Informem o período analisado e definam o que será considerado uma entrega concluída.

Abordagem A – Entregas funcionais por hora:

Produtividade = Número de entregas concluídas ÷ Horas de desenvolvimento

Exemplo: 12 entregas funcionais concluídas em 120 horas correspondem a 0,1 entrega por hora. A equipe deve definir o que caracteriza uma entrega e considerar que funcionalidades podem ter complexidades diferentes.

Abordagem B – Story Points por sprint (se a equipe já utiliza estimativas ágeis):

Resultado = Story Points concluídos na sprint

Os Story Points representam uma estimativa relativa de esforço/complexidade definida pela própria equipe. São úteis para acompanhar a evolução da equipe ao longo das sprints, mas não devem ser usados para comparar diretamente equipes diferentes.

Indicador complementar – Linhas de código por hora (usar com ressalvas):

LOC/hora = Linhas de código produzidas no período ÷ Horas de desenvolvimento no mesmo período

Só utilizem esse cálculo se conseguirem identificar as linhas efetivamente produzidas no período e as horas correspondentes. Não dividam o tamanho total acumulado do sistema pelas horas de um período menor. LOC/hora não mede, por si só, valor entregue, qualidade ou eficiência.

Conexão com o negócio: a análise deve ajudar a discutir a capacidade de entrega e o planejamento do roadmap, sem transformar uma métrica isolada em avaliação individual de desempenho.

5. Relação entre retrabalho e custo financeiro

Para traduzir o retrabalho em impacto financeiro, a equipe deverá estimar o custo das horas classificadas como retrabalho. O custo-hora pode ser real, quando disponível, ou simulado para fins didáticos. Informem a premissa utilizada.

Custo estimado do retrabalho = Horas de retrabalho × Custo estimado por hora

Exemplo: 45 horas de retrabalho × R$ 40,00 por hora = R$ 1.800,00. O resultado é uma estimativa e depende da qualidade dos dados de horas e do valor de custo-hora adotado.

6. Roteiro sugerido para o pitch executivo

Na próxima aula, cada equipe terá até 10 minutos para apresentar os resultados, seguidos de até 3 minutos de arguição. A apresentação deverá ter, preferencialmente, até cinco slides:

00 a 02 min – Slide 1: Visão geral do PI e arquitetura

Apresentar o problema de negócio, o objetivo do sistema, a stack tecnológica e uma visão do escopo (módulos, funcionalidades e tamanho do código, com critério de contagem informado).

02 a 05 min – Slides 2 e 3: As quatro métricas

Exibir os dados coletados, os cálculos e os resultados de densidade de defeitos, cobertura de testes, taxa de retrabalho e produtividade. Informar período, fonte dos dados e limitações. Comparações externas só devem ser feitas com referências contextualizadas.

05 a 08 min – Slide 4: Impacto no plano de negócios

Relacionar os resultados a riscos técnicos e operacionais, custo estimado do retrabalho, experiência do usuário e capacidade de evolução do produto. Evitar afirmar que uma métrica isolada comprova churn ou escalabilidade.

08 a 10 min – Slide 5: Plano de ação e próximos passos

Apresentar três ações concretas, priorizadas e viáveis para as próximas sprints, indicando o problema que cada ação pretende reduzir e como será acompanhado o resultado.

Atenção ao tempo: o limite de 10 minutos será considerado na avaliação. Ensaiem a apresentação e distribuam as falas entre os integrantes.

7. Quadro de diagnóstico técnico da equipe

Preencham o quadro durante a aula. Registrem os valores brutos, o resultado calculado e um diagnóstico inicial baseado nas evidências. Quando um dado não estiver disponível, indiquem “não disponível” e expliquem como pretendem obtê-lo ou estimá-lo.

Item/métrica do PI	Valor bruto coletado	Resultado da métrica	Diagnóstico inicial/evidências
Nome da startup/Projeto Integrador	Nome da solução	—	Contexto e objetivo do projeto
Link do repositório	URL ou indicação de acesso docente	—	Repositório utilizado e data da coleta
Período analisado	Data inicial e final	—	Justificativa do período
Tamanho do sistema (LOC/KLOC)	Ex.: 4.800 LOC	Ex.: 4,8 KLOC	Critério de contagem e arquivos incluídos/excluídos
1. Densidade de defeitos	Ex.: 24 defeitos registrados	Ex.: 5 defeitos/KLOC	Severidade, ambiente e limitações
2. Cobertura de testes	Ex.: 2.400 linhas cobertas / 4.800 consideradas	Ex.: 50%	Tipo de cobertura e interpretação
3. Taxa de retrabalho	Ex.: 45 h / 180 h	Ex.: 25%	Critério de classificação e fonte das horas
4. Produtividade	Entregas, Story Points ou LOC e horas	Resultado conforme abordagem escolhida	Período, complexidade e limitações
Custo estimado do retrabalho	Horas de retrabalho e custo-hora	Ex.: R$ 1.800,00	Premissas usadas no cálculo

8. Critérios de avaliação do pitch

A apresentação será avaliada considerando a correção dos cálculos, a interpretação dos resultados e a capacidade de propor decisões fundamentadas.

Critério	O que será observado
Correção dos cálculos	Dados, fórmulas, unidades e coerência dos resultados.
Interpretação dos resultados	Capacidade de explicar o significado e as limitações das métricas.
Relação entre qualidade e negócio	Análise de impactos técnicos, operacionais e financeiros.
Plano de ação	Ações específicas, viáveis e relacionadas aos problemas identificados.
Apresentação e domínio	Clareza, organização, participação da equipe e respeito ao tempo.

9. Checklist para a próxima aula

☐ Repositório e fontes de dados identificados; período e critérios de coleta registrados.

☐ Métricas calculadas e conferidas, com dados reais ou estimativas devidamente identificadas.

☐ Limitações dos indicadores explicadas.

☐ Horas de retrabalho convertidas em custo estimado em reais (R$), com premissas informadas.

☐ Apresentação organizada em até cinco slides objetivos.

☐ Três ações de melhoria definidas para as próximas sprints.

☐ Tempo total ensaiado para não ultrapassar 10 minutos.

☐ Todos os integrantes preparados para responder aos questionamentos.

10. Observações finais

Caso o Projeto Integrador ainda não esteja concluído, utilizem os dados disponíveis até a data da coleta. Não inventem valores para preencher o quadro: quando necessário, apresentem estimativas identificadas, expliquem as premissas e indiquem quais informações ainda precisam ser coletadas.

Referências e materiais de consulta

As equipes devem consultar a documentação oficial das ferramentas utilizadas e fontes técnicas confiáveis. Sugestões iniciais:

- ISTQB. Certified Tester Foundation Level (CTFL) – syllabus. Consultar a versão vigente no portal oficial do ISTQB.
- Google. Testing on the Toilet / Testing Blog e documentação de ferramentas de teste, conforme a tecnologia utilizada.
- Documentação oficial do pytest: https://docs.pytest.org/
- Documentação oficial do Jest: https://jestjs.io/docs/
- Documentação oficial do PHPUnit: https://docs.phpunit.de/
- GitHub Docs: documentação sobre issues, pull requests e métricas disponíveis no repositório: https://docs.github.com/

Observação: as referências acima são pontos de partida para consulta. A equipe deverá citar, nos slides, as fontes efetivamente utilizadas e não atribuir a essas fontes benchmarks que elas não apresentem.